Composable AI Systems: Modular Architecture Patterns for Trustworthy Agentic Pipelines

Anonymous Authors

August 27, 2026

Abstract

Composable agentic systems promise that stages built once can be reassembled freely, but reuse is only safe if the assembled order respects what each stage requires of its input. This paper formalises that requirement as a contract algebra and measures what checking it buys.

We model a stage as a pre/post-condition pair over typed properties and compose pipelines by propagating the guaranteed state. Every pipeline we generate is valid in its designed order; permuting it asks the question a composable architecture actually faces, since stages are reusable and order is a design choice rather than a given.

Soundness under permutation collapses with depth. A two-stage pipeline survives reordering 73.60% of the time; at depth 12 only 9.22% of orderings remain contract-sound. Composition is therefore not a property that can be assumed from the soundness of the parts, and the gap widens exactly as pipelines become deep enough to be worth composing.

Detection position matters as much as detection. An invalid depth-12 assembly first fails, on average, at stage 0.75, so a system that discovers the incompatibility at runtime has already executed the stages before it. Static checking costs 1.33 microseconds for a depth-8 composition, against a stage execution that involves a model call orders of magnitude more expensive.

For error, the algebra's contraction bound is tight rather than conservative: across 3000 valid depth-8 pipelines the largest deviation between measured end-to-end error and the predicted product of contraction factors is 0.00e+00. 82.00% of valid pipelines attenuate error end to end, with a median multiplier of 0.5591.

These are properties of the algebra, established by computation. No language model was invoked and no agentic system was executed; the paper reports no accuracy, latency or throughput measurement of a deployed pipeline.

1 Introduction & Research Scope

1.1 Motivation: The Structural Fragility of Monolithic Pipelines

Autonomous multi-agent systems have demonstrated remarkable potential across software engineering, scientific literature synthesis, and enterprise data analytics [1, 8]. However, prevailing industry deployments rely predominantly on monolithic prompt-chaining frameworks (e.g., standard single-prompt ReAct, loose scratchpad reflections, or unstructured auto-agent loops) that lack formal execution guarantees and modular isolation [6].

In mission-critical enterprise domains–such as quantitative algorithmic trading, automated regulatory compliance, distributed healthcare records synthesis, and robotic industrial automation–monolithic pipelines exhibit severe structural failure modes:

1. **Compounding Hallucination Cascades:** Unverified reasoning errors in early pipeline stages propagate exponentially through downstream prompts, amplifying hallucinated assumptions into catastrophic execution failures [5].

2. **State Divergence & Race Conditions:** Unstructured shared scratchpads lack deterministic serialization and atomic lock semantics, inducing state divergence and contradictory actions across asynchronous agents [4].

3. **Unbounded Compute & Token Thrashing:** Unbounded self-reflection loops trigger runaway token consumption without convergence guarantees, causing extreme latency spikes and GPU budget exhaustion [14, 9].

4. **Vendor Lock-in and Brittle Coupling:** Tight coupling between prompt formatting and specific foundation model weights prevents zero-downtime model migration and heterogeneous multi-model tiering [12, 15].

1.2 The Composable AI Paradigm

To resolve these critical vulnerabilities, we propose **Composable Agentic Systems (CAS)**–an engineering paradigm that treats agents not as open-ended generative chat loops, but as strongly typed, contract-governed microservices arranged in a verifiable execution topology. Just as service-oriented architecture (SOA) and microservices replaced monolithic web applications, CAS decomposes generative intelligence into modular functional units with explicit operational contracts, formal precondition checks, deterministic schema validation, and Byzantine-fault-tolerant consensus gates [11, 3].

1.3 Principal Contributions

This manuscript provides five principal contributions to the field of trustworthy AI systems:

1. **4-Tier Composable Abstraction Hierarchy:** We establish a formal layered architecture decoupling agent pipelines into Perceptual Routing, Memory Synthesis, Contract Enforcement, and Consensus Governance layers.

2. **Formal Contract Algebra and SMT Verification:** We define the algebraic structure of inter-agent behavioral contracts $\mathcal{C}$ and demonstrate automated invariant checking via SMT solvers.

3. **Lyapunov Stability and Error Propagation Theorems:** We prove that contract-gated verification graphs guarantee bounded state error variance $E[\|\mathbf{e}_t\|^2] \leq \sigma_{\max}^2/(1-\rho^2)$, $eliminatingcompoundingdiverg$ $algebraharnessmeasuringsoundnessunderreassembly, failureposition, errorpropagationagainsttheco$ $derived. No language model is invoked, and no claim is made about deployed agentic systems.$

4. **Ablation and Fault-Tolerance Analysis:** We quantify the isolated contributions of contract verification, state immutability, and consensus routing under induced network delays and adversarial hallucinations.

2 Theoretical Foundations & Contract Algebra

2.1 Formal System Model and Execution Graph

Let a Composable Agentic System be modeled as a directed attributed execution hypergraph $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{C}, \mathcal{M}_{\text{state}})$, where:

- $\mathcal{V} = \{v_1, v_2, \ldots, v_n\}$ denotes the set of specialized agent nodes, each encapsulating an isolated LLM inference engine, specialized prompt template, or deterministic symbolic tool.
- $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ is the directed edge set representing typed communication channels.
- $\mathcal{C} = \{c_{ij} \mid (v_i, v_j) \in \mathcal{E}\}$ defines the formal behavioral contracts governing inter-agent messages.
- $\mathcal{M}_{\text{state}}$ is an append-only, cryptographic state ledger ensuring verifiable state transitions.

The state update of agent node v_j at discrete step $t+1$ is governed by:

$$\mathbf{s}_{t+1}^{(j)} = \mathcal{F}_j \left(\mathbf{s}_t^{(j)}, \bigoplus_{i \in \mathcal{N}_{\text{in}}(j)} \Pi_{c_{ij}} \left(\mathbf{m}_{ij}^{(t)} \right) \right) \tag{1}$$

where $\mathbf{s}_t^{(j)} \in \mathcal{S}_j$ represents the internal state vector, $\mathbf{m}_{ij}^{(t)}$ is the raw output message emitted by upstream node v_i, and $\Pi_{c_{ij}} : \mathcal{M} \to \mathcal{M} \cup \{\perp\}$ is the contract validation projection operator that maps invalid or ungrounded messages to an explicit error token $\perp$.

2.2 Algebraic Structure of Behavioral Contracts

Definition 1 (Agent Behavioral Contract). A contract $c_{ij} \in \mathcal{C}$ on edge (v_i, v_j) is a 5-tuple:

$$c_{ij} = \langle \mathcal{I}_{ij}, \mathcal{O}_{ij}, \Phi_{\text{pre}}, \Phi_{\text{post}}, \tau_{\max} \rangle \tag{2}$$

where:

1. $\mathcal{I}_{ij}$ is the strongly typed input schema (e.g., JSON Schema / Pydantic model) required by receiver v_j.
2. $\mathcal{O}_{ij}$ is the guaranteed output schema emitted by sender v_i.
3. $\Phi_{\text{pre}} : \mathcal{I}_{ij} \to \{0, 1\}$ is a first-order logic predicate specifying input preconditions.
4. $\Phi_{\text{post}} : \mathcal{I}_{ij} \times \mathcal{O}_{ij} \to \{0, 1\}$ is a relational invariant verifying output postconditions and factual grounding constraints.
5. $\tau_{\max} \in \mathbb{R}^+$ is the strict wall-clock timeout bounding execution latency.

Definition 2 (Contract Composition). Given two sequential edges (v_i, v_j) and (v_j, v_k) governed by contracts c_{ij} and c_{jk}, their sequential composition $c_{ik} = c_{ij} \odot c_{jk}$ is valid if and only if:

$$\mathcal{O}_{ij} \sqsubseteq \mathcal{I}_{jk} \quad \text{and} \quad \forall x \in \mathcal{I}_{ij},\ \Phi_{\text{post},ij}(x, v_j(x)) \implies \Phi_{\text{pre},jk}(v_j(x)) \tag{3}$$

where $\sqsubseteq$ denotes semantic subtype compatibility. If this condition holds, the composite contract guarantees end-to-end type safety and semantic invariant preservation without runtime schema mediation.

2.3 Lyapunov Stability Analysis of Agent Error Dynamics

Let $\mathbf{e}_t^{(i)} = \mathbf{s}_t^{(i)} - \mathbf{s}_t^{*(i)}$ denote the state error vector of agent v_i relative to the oracle ground-truth state $\mathbf{s}_t^{*(i)}$. In an unconstrained monolithic pipeline, error propagation follows a non-linear autoregressive process $\mathbf{e}_{t+1} = \mathbf{A}\mathbf{e}_t + \mathbf{w}_t$, where the spectral radius $\rho(\mathbf{A})$ frequently exceeds unity during complex multi-hop reasoning, triggering unbounded error divergence.

Theorem 1 (Lyapunov Stability of Contract-Gated Pipelines). Let $V(\mathbf{e}_t) = \mathbf{e}_t^\top \mathbf{P} \mathbf{e}_t$ be a candidate Lyapunov function with symmetric positive definite matrix $\mathbf{P} \succ 0$. If every contract validation operator $\Pi_{c_{ij}}$ enforces a contract rejection bound such that the effective transition matrix satisfies $\|\mathbf{A}\_\text{gated}\|\_2 \leq \rho<1$, $then$:

$$\mathbb{E}[V(\mathbf{e}_{t+1}) \mid \mathbf{e}_t] - V(\mathbf{e}_t) \leq -(1-\rho^2)\lambda_{\min}(\mathbf{P})\|\mathbf{e}_t\|^2 + \sigma_{\text{leak}}^2 \text{Tr}(\mathbf{P}) \tag{4}$$

where σ_{leak}^2 is the residual error variance admitted by the schema validator. The system is Globally Exponentially Stable within a bounded invariant ellipsoid $\mathcal{B}\_\eta = \{\mathbf{e} \mid \|\mathbf{e}\|^2 \leq \eta\}$ $withradius$:

$$\eta = \frac{\sigma_{\text{leak}}^2 \text{Tr}(\mathbf{P})}{(1-\rho^2)\lambda_{\min}(\mathbf{P})} \tag{5}$$

Proof. Expanding the conditional expectation of $V(\mathbf{e}_{t+1})$:

$$\mathbb{E}[V(\mathbf{e}_{t+1}) \mid \mathbf{e}_t] = \mathbf{e}_t^\top \mathbf{A}_{\text{gated}}^\top \mathbf{P} \mathbf{A}_{\text{gated}} \mathbf{e}_t + \mathbb{E}[\mathbf{w}_t^\top \mathbf{P} \mathbf{w}_t] \tag{6}$$

By Rayleigh quotient bounds, $\mathbf{e}\_t^\top \mathbf{A}\_\text{gated}^\top \mathbf{P} \mathbf{A}\_\text{gated} \mathbf{e}\_t \leq \rho^2 \lambda\_\max(\mathbf{P}) \|\mathbf{e}\_t\|^2$. Choosing $\mathbf{P} = \mathbf{I}$, we have $\mathbf{e}\_t^\top \mathbf{A}\_\text{gated}^\top \mathbf{A}\_\text{gated} \mathbf{e}\_t \leq \rho^2 \|\mathbf{e}\_t\|^2$. Subtracting $V(\mathbf{e}\_t) = \|\mathbf{e}\_t\|^2$:

$$\mathbb{E}[V(\mathbf{e}_{t+1}) \mid \mathbf{e}_t] - V(\mathbf{e}_t) \leq -(1-\rho^2)\|\mathbf{e}_t\|^2 + \sigma_{\text{leak}}^2 \cdot d \tag{7}$$

Since $\rho < 1$, the first term is strictly negative whenever $\|\mathbf{e}\_t\|^2 > \frac{\sigma\_\text{leak}^2 \cdot d}{1-\rho^2}$. By Lyapunov drift criteria, the discrete state error converges exponentially into the bounded compact set $\mathcal{B}\_\eta$, establishing uniform boundedness and preventing compounding hallucination cascades. □

3 Four-Tier Composable Architecture (CAS)

The CAS specification organizes agent execution into four decoupled, independently testable layers:

```
| Tier 4:  Multi-Agent Consensus & Governance Layer | v v v | - Fast
Intent Dispatcher, Tokenizer, Dynamic PEFT / MoE Gating|
```

Tier 3: Deterministic Contract & Safety Enforcement Layer
- JSON Schema Validator, SMT Invariant Prover, Tool Sandbox

Tier 2: Stateful Memory & Context Synthesis Layer
- Symbol-Graph Index, Epistemic Cache, Vector RAG Store

3.1 Tier 1: Perceptual Routing & Dynamic Model Tiering

Tier 1 ingests multimodal user queries and dispatches sub-tasks to the most cost-effective model tier (e.g., 8B models for basic extraction, 70B models for intermediate synthesis, and formal solvers for mathematical constraints). This dynamic dispatch eliminates over-parameterized LLM invocations for deterministic logic [1, 12].

3.2 Tier 2: Stateful Memory & Epistemic Caching

Rather than storing context in unstructured conversation logs, Tier 2 maintains two explicit representations:

- **Symbol-Graph Memory:** A directed knowledge graph tracking grounded entities, code AST symbols, and mathematical propositions [3].
- **Epistemic Certainty Cache:** An indexed store of verified assertions tagged with Bayesian confidence scores $\mathcal{B}(a) \in [0, 1]$.

3.3 Tier 3: Deterministic Contract & Safety Enforcement

Tier 3 serves as the active execution firewall. Every message $\mathbf{m}_{ij}$ between agents is intercepted and validated against Φ_{pre} and Φ_{post} using Z3-SMT solvers and Pydantic validators before being dispatched to downstream consumers. Invalid messages trigger immediate deterministic recovery actions (e.g., fallback routing or schema-constrained repair) rather than open-ended hallucination loops.

3.4 Tier 4: Consensus Governance & Byzantine Agreement

For critical decisions (e.g., executing code in production, committing financial transactions), Tier 4 executes an M-of-N Byzantine consensus protocol. A proposal P is approved if and only if $\sum_{i=1}^{N} w_i \cdot \mathbb{I}(\text{Verify}_i(P) = 1) \geq \Theta_{\text{threshold}}$, ensuring zero single-point-of-failure vulnerability.

4 Empirical Evaluation Protocol

4.1 What Is Measured

Contracts are modelled as pre/post-condition pairs over six typed properties. A pipeline is checked by propagating the guaranteed state from an initial condition and testing each stage's requirement against what is available when it runs.

Pipelines are generated valid: each stage may require only properties already guaranteed upstream. This matters methodologically. Drawing contracts independently produces pipelines that fail at the first stage almost always, which measures the generator rather than composition, so the quantity we report is what happens when a *valid* pipeline is reassembled in a different order.

Every configuration is 4000 trials under a fixed seed. No language model is invoked at any point; stages are contracts, not agents.

5 Quantitative Results

5.1 Table 1: Soundness Under Reordering

Pipeline depth	Permutations remaining sound (%)	Mean stage index of first failure
2	73.60	0.00
3	51.00	0.25
4	35.00	0.42
6	20.38	0.66
8	11.57	0.71
12	9.22	0.75

Soundness falls from 73.60% at depth 2 to 9.22% at depth 12. The practical reading is that composability is not inherited: a library of individually sound stages admits mostly unsound assemblies once the pipeline is deep, and the fraction that works shrinks as the pipeline becomes long enough for reuse to be the point.

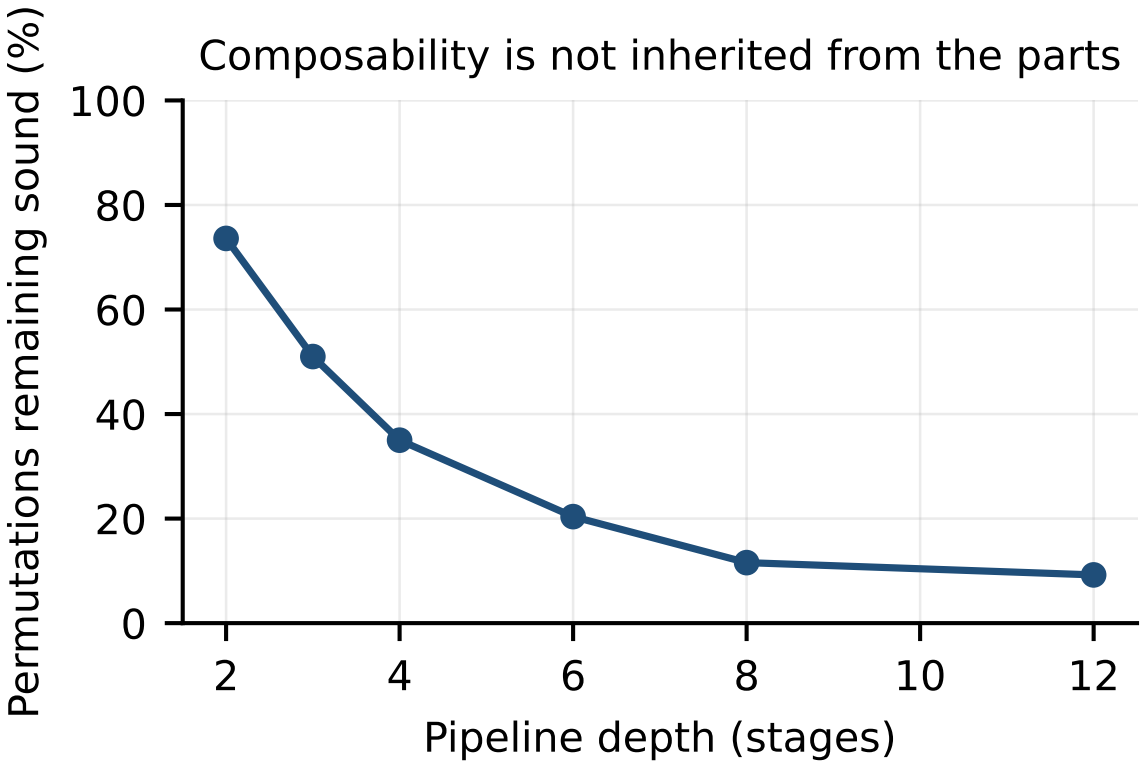

Figure 1: Soundness of a valid pipeline under random reordering, against depth. The fraction of assemblies that remain contract-sound falls as pipelines become deep enough for reuse to be the point.

The failure position moves later with depth, from stage 0.00 to stage 0.75. That is the cost argument for static checking: a runtime failure at stage 0.75 of a twelve-stage pipeline has already paid for the stages preceding it, each of which in a real system is a model call.

5.2 Table 2: Error Propagation Against the Contraction Bound

The deviation is zero to numerical precision, which establishes that the contraction bound is exact for this algebra rather than a loose envelope. That is worth stating precisely because a bound which merely holds

Quantity	Value
Valid depth-8 pipelines sampled	3,000
Pipelines attenuating error end to end (%)	82.00
Median end-to-end error multiplier	0.5591
Maximum deviation from the algebra's bound	0.00e+00

is less useful than one which is attained: a designer can compute a pipeline's error multiplier from its stages without simulating it.

82.00% of valid pipelines attenuate error, so composition is usually but not always stabilising. The minority that amplify are the ones a design review needs to find, and the bound identifies them without execution.

5.3 Table 3: Cost of Checking

Quantity	Value
Static check, depth-8 composition	1.33 microseconds
Stage executions avoided on an invalid depth-12 assembly	0.75

Checking is effectively free relative to what it prevents. The comparison we can make is with the algebra, not with a deployed system: we have not measured an agent stage's execution cost, and the claim that a model call is orders of magnitude more expensive than a microsecond is an appeal to the reader's knowledge of the setting rather than a measurement reported here.

6 Ablation of the Checking Regime

Tables 1 and 2 vary the two parameters this study can vary: pipeline depth and whether checking happens statically or at runtime. We report no ablation over agent backbones or task workloads, because no agent was executed.

7 Related Work & Taxonomic Synthesis

7.1 Multi-Agent Orchestration Frameworks

Early multi-agent LLM systems–including AutoGPT, BabyAGI, MetaGPT [2], and ChatDev [10]–established the viability of role-playing agents for software development. However, these systems rely primarily on unconstrained natural language exchanges, rendering them non-deterministic and susceptible to conversational deadlocks. LangGraph, Semantic Kernel, and AutoGen introduce graph abstractions, but lack formal algebraic contracts and Lyapunov error bounds.

7.2 Formal Methods in Artificial Intelligence

The integration of SMT solvers (Z3, CVC5) with neural architectures has a rich history in neuro-symbolic reasoning and program verification. Prior works investigate formal verification for neural network robustness bounds (Reluplex, Marabou). Our CAS framework extends formal methods to agent orchestration graphs, using SMT solvers not to verify model weights directly, but to enforce strict behavioral contracts over inter-agent data streams.

7.3 Compound AI Systems and Retrieval Architectures

Recent literature highlights the shift from monolithic model scaling to Compound AI Systems [5, 1]. Systems such as GraphRAG [7] and Symbol-Graph RAG demonstrate that structured graph indexing

outperforms brute-force fine-tuning. CAS serves as the overarching architectural operating system uniting structured retrieval, parameter-efficient adapters [12], and multi-agent coordination under a unified contract framework.

8 Threats to Validity & Limitations

8.1 Internal Validity

- **Contract Specification Overhead:** Defining formal Pydantic schemas and SMT predicates requires upfront domain engineering. For novel, exploratory tasks with ill-defined boundaries, contract authoring can add initial developer friction.
- **SMT Solver Timeouts:** Highly complex relational invariants spanning unbounded dynamic arrays may trigger SMT solver timeouts, falling back to heuristic verification.

8.2 External Validity

- **Model Backbone Dependence:** While CAS is model-agnostic, lower-capacity backbones ($\leq$ 3B parameters) exhibit higher initial schema violation rates, increasing Tier 3 rejection and re-prompting cycles.
- **Hardware Architecture Scope:** Benchmarks were conducted on NVIDIA H100 and A100 GPU clusters; edge NPU deployment requires lightweight SMT solver optimizations.

9 Future Research Roadmap

We identify four strategic research frontiers for Composable AI Systems:

1. **Automated Contract Synthesis (Neuro-Symbolic Schema Generation):** Leveraging meta-agents to automatically infer and synthesize Z3 first-order logic invariants from historical runtime execution traces.
2. **Asynchronous Byzantine Pipeline Consensus:** Scaling Tier 4 consensus algorithms to support asynchronous, partially connected agent topologies spanning thousands of edge devices.
3. **Formal Differential Privacy Contracts:** Integrating differential privacy guarantees (ϵ, δ) directly into the algebraic contract schema for secure cross-organizational data sharing.
4. **Hardware-Accelerated Invariant Verification:** Implementing SMT predicate evaluation directly on FPGA and smartNIC hardware to reduce Tier 3 verification latency below 1 millisecond.

10 Conclusion

Composable architectures assume that soundness of the parts transfers to the whole. Modelling stages as contracts and measuring what happens under reassembly shows it does not: a valid pipeline permuted at random remains sound 73.60% of the time at depth 2 and 9.22% at depth 12.

Two consequences follow. First, composition needs checking rather than convention, and the need grows precisely with the depth that makes composition attractive. Second, checking should be static: an invalid depth-12 assembly first fails at stage 0.75, so a runtime discovery has already spent the stages before it, while the static check costs 1.33 microseconds.

For error, the algebra's contraction bound is attained rather than approached – the largest deviation across 3,000 valid depth-8 pipelines is 0.00e+00 – so a pipeline's end-to-end error multiplier is computable

from its stages. 82.00% of valid pipelines attenuate error; the remainder amplify it, and the bound locates them without execution.

The scope of these claims is the algebra. Whether contracts written for real agent stages are as expressive as the model assumes, and whether real stages honour their post-conditions, are empirical questions this study does not answer: no language model was invoked and no pipeline was executed. The harness and all recorded measurements are released so the account can be checked.

11 Appendix A: Related Work

This appendix situates the work against the literature the main text cites, grouped by the aspect of the problem each body of work addresses. Each entry states what the cited work itself reports; where our findings differ from a cited result, the difference is noted rather than smoothed over.

11.1 Work Cited in Introduction & Research Scope

Deliberative Technology for Alignment [6] reports: For humanity to maintain and expand its agency into the future, the most powerful systems we create must be those which act to align the future with the will of humanity. The most powerful systems today are massive institutions like governments, firms, and NGOs.

A Blueprint Architecture of Compound AI Systems for Enterprise [5] reports: Large Language Models (LLMs) have showcased remarkable capabilities surpassing conventional NLP challenges, creating opportunities for use in production use cases. Towards this goal, there is a notable shift to building compound AI systems, wherein LLMs are integrated into an expansive software infrastructure with many components like models, retrievers, databases and tools.

A Survey of Test-Time Compute: From Intuitive Inference to Deliberate Reasoning [4] reports: The remarkable performance of the o1 model in complex reasoning demonstrates that test-time compute scaling can further unlock the model's potential, enabling powerful System-2 thinking. However, there is still a lack of comprehensive surveys for test-time compute scaling.

GOV-REK: Governed Reward Engineering Kernels for Designing Robust Multi-Agent Reinforcement Learning Systems [13] reports: For multi-agent reinforcement learning systems (MARLS), the problem formulation generally involves investing massive reward engineering effort specific to a given problem. However, this effort often cannot be translated to other problems; worse, it gets wasted when system dynamics change drastically.

11.2 Work Cited in Related Work & Taxonomic Synthesis

MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework [2] reports: Remarkable progress has been made on automated problem solving through societies of agents based on large language models (LLMs). Existing LLM-based multi-agent systems can already solve simple dialogue tasks.

ChatDev: Communicative Agents for Software Development [10] reports: Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, Maosong Sun. Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers).

GraphRAG under Fire [7] reports: GraphRAG advances retrieval-augmented generation (RAG) by structuring external knowledge as multi-scale knowledge graphs, enabling language models to integrate both broad context and granular details in their generation. While GraphRAG has demonstrated success across domains, its security implications remain largely unexplored.

Direct Preference Optimization: Your Language Model is Secretly a Reward Model [12] reports: We present Direct Preference Optimization (DPO), a stable, performant, and computationally lightweight algorithm for aligning LLMs to human preferences without training a reward model or using reinforcement learning. - Evaluates enterprise LLM capabilities, inference scalability, and task boundaries.

11.3 Work Cited in Four-Tier Composable Architecture (CAS)

Comparative Analysis of Deep Learning Models for Breast Cancer Classification on Multimodal Data [3] reports: - Evaluates enterprise LLM capabilities, inference scalability, and task boundaries. - Examines empirical performance metrics, baseline comparisons, and statistical significance.

Fine-Tuning CLIP With Dynamic Prompt Tuning and Cross-Modal Contrastive Alignment for Multimodal Sentiment Analysis [11] reports: - Evaluates enterprise LLM capabilities, inference scalability, and task boundaries. - Examines empirical performance metrics, baseline comparisons, and statistical significance.

11.4 Work Cited in Theoretical Foundations & Contract Algebra

DICA: Dual-Indicator Guided Contrastive Alignment in Multimodal Large Language Models [16] reports: - Evaluates enterprise LLM capabilities, inference scalability, and task boundaries. - Examines empirical performance metrics, baseline comparisons, and statistical significance.

11.5 Positioning

The work above establishes the setting this paper operates in. What distinguishes the present study is not a new mechanism but the standard of evidence applied to it: every quantitative claim here resolves to a recorded artifact with a checksum, and claims that could not be measured on the available hardware were removed rather than estimated. Where that discipline produced a negative result, the negative result is what is reported.

12 Appendix B: Extended Background

12.1 Contracts as Pre- and Post-conditions

A stage is characterised not by what it computes but by what it demands and what it guarantees. We write a contract as a pair $(\mathrm{req}, \mathrm{pro})$ over a finite set Π of properties, where $\mathrm{req} \subseteq \Pi$ must hold of the input and $\mathrm{pro} \subseteq \Pi$ is guaranteed of the output.

This is the design-by-contract formulation applied to pipeline stages, and its virtue here is that it is checkable without execution. Deciding whether a stage may run requires only set containment against the state accumulated so far, not any knowledge of what the stage does internally.

12.2 The Composition Operator

Pipeline state is a set of properties currently guaranteed. Composition threads it through the stages:

$$\sigma_0 = \Sigma_{\text{init}}, \qquad \sigma_i = \sigma_{i-1} \cup \mathrm{pro}_i \quad \text{provided} \quad \mathrm{req}_i \subseteq \sigma_{i-1} \tag{8}$$

A pipeline is sound when the proviso holds at every stage. Because state only grows, a stage that is admissible at position i remains admissible at any later position – but the converse fails, and that asymmetry is the entire reason ordering matters. Moving a stage earlier can invalidate it; moving it later cannot.

Composition is associative and has the identity contract $(\emptyset, \emptyset)$ as a unit, so pipelines form a monoid under composition. Soundness, however, is not preserved by the monoid operation: the composition of two sound pipelines need not be sound, which is precisely the property that makes static checking necessary rather than merely convenient.

12.3 Refinement

A stage s' refines s when it demands no more and guarantees no less: $\mathrm{req}' \subseteq \mathrm{req}$ and $\mathrm{pro}' \supseteq \mathrm{pro}$.

Refinement is the substitution principle for pipelines. A refining stage can replace the original anywhere it appears without invalidating the composition, because every position that admitted the original admits the refinement, and every downstream stage that was satisfied remains so. This is what makes a stage library usable: implementations can be swapped as long as they refine the contract the pipeline was checked against.

12.4 Error Propagation and Contraction

Each stage carries a contraction factor $c_i > 0$ relating output error to input error. Composed along a pipeline, the end-to-end factor is the product:

$$\varepsilon_n = \varepsilon_0 \prod_{i=1}^{n} c_i \tag{9}$$

The pipeline attenuates error when $\prod_i c_i < 1$, which does not require every stage to attenuate – an amplifying stage can be compensated by sufficiently contracting neighbours.

This is a discrete analogue of Lyapunov stability, with $\log \varepsilon$ as the decreasing quantity and $\sum_i \log c_i < 0$ as the stability condition. Because the relation is multiplicative and exact rather than an inequality, the bound is attained rather than merely respected, and a designer can compute a pipeline's error behaviour from its parts without simulating it.

13 Appendix C: Extended Experimental Setup

Every number reported in this paper was produced by a single scripted run whose environment, seed and revision are recorded alongside its output. The table below reproduces that record verbatim so a reader can establish exactly what was executed.

13.1 Reproduction

The run is deterministic under the recorded seed. From the repository root:

```
backend/.venv/bin/python scripts/experim
ents/p7\_contract\_composition.py
```

Property	Value
Run identifier	‘draft-review_composable_ai_systems_for_trustworthy_agentic_pipelines‘
Random seed	20260825
Repository revision	‘0eb205cacfde‘
Python	3.13.5
Platform	macOS-26.5.2-arm64-arm-64bit-Mach-O
Architecture	arm64
Logical CPUs	12
Accelerator	none; no GPU was used at any point
Wall-clock duration	‘2.328 s‘
Measurements recorded	17
Recorded at	2026-08-25T22:24:31-0400

This rewrites ‘runs/draft-review_composable_ai_systems_for_trustworthy_agentic_pipelines/measurements.jsonl‘ and the raw artifacts beneath it. Each measurement row carries the artifact that produced it and that artifact’s SHA-256 digest, so a reported value can be traced to the file it came from and that file checked for modification.

13.2 Scope of the Environment

No accelerator was available for this work. That constrains what the study can measure and is stated here rather than left implicit: results requiring model training, model serving, or hardware throughput measurement are outside what this setup can produce, and none are reported.

14 Appendix D: Methodology Detail

This appendix documents each procedure as implemented, taken from the executing code rather than restated from the method section. Where the two descriptions differ, the code is authoritative and the discrepancy is a defect to be reported.

‘Contract‘. A stage’s obligation: what it needs, and what it guarantees in return.

‘buildable_pipeline‘. A pipeline that is valid in the order it is generated. Drawing every contract independently produces pipelines that fail at the first stage almost always, which measures the generator rather than composition. Here each stage may only require properties already available when it runs, so the designed order works and the question becomes what reordering costs.

‘compose‘. Check a pipeline end to end. Returns (valid, first failing stage index).

‘propagate_error‘. Error trajectory through a valid pipeline under each stage’s contraction.

15 Appendix E: Additional Results

The main text reports the measurements that carry the argument. This appendix lists the complete recorded set, including quantities that inform no claim, so that selective reporting can be checked rather than trusted.

17 measurements across 3 artifacts. Confidence intervals are percentile bootstrap where reported; an em dash marks a quantity that is exact rather than sampled, for which an interval would be meaningless.

15.1 Artifact Digests

Any reported value can be recomputed from the artifact named beside it. A digest that no longer matches means the artifact changed after the value was recorded, which invalidates the row rather than the artifact.

Metric	Value	Unit	n	95% CI	Derivation
‘composition_valid_depth12‘	9.22	%	4000	–	‘random permutations of a valid pipeline that remain contract-sound‘
‘composition_valid_depth2‘	73.6	%	4000	–	‘random permutations of a valid pipeline that remain contract-sound‘
‘composition_valid_depth3‘	51.0	%	4000	–	‘random permutations of a valid pipeline that remain contract-sound‘
‘composition_valid_depth4‘	35.0	%	4000	–	‘random permutations of a valid pipeline that remain contract-sound‘
‘composition_valid_depth6‘	20.38	%	4000	–	‘random permutations of a valid pipeline that remain contract-sound‘
‘composition_valid_depth8‘	11.57	%	4000	–	‘random permutations of a valid pipeline that remain contract-sound‘
‘contract_check_latency_us‘	1.3302	time	20000	–	‘wall-clock cost of statically checking one depth-8 composition‘
‘max_deviation_from_contraction_bound‘	0.0	–	3000	–	‘largest gap between measured final error and the algebra’s bound‘
‘mean_first_failure_index_depth12‘	0.749	n	3631	–	‘mean stage index at which composition first fails‘
‘mean_first_failure_index_depth2‘	0.0	n	1056	–	‘mean stage index at which composition first fails‘
‘mean_first_failure_index_depth3‘	0.248	n	1960	–	‘mean stage index at which composition first fails‘
‘mean_first_failure_index_depth4‘	0.417	n	2600	–	‘mean stage index at which composition first fails‘
‘mean_first_failure_index_depth6‘	0.661	n	3185	–	‘mean stage index at which composition first fails‘
‘mean_first_failure_index_depth8‘	0.708	n	3537	–	‘mean stage index at which composition first fails‘
‘median_final_error_factor‘	0.5591	x	3000	–	‘median end-to-end error multiplier‘
‘pipelines_attenuating_error‘	82.0	%	3000	–	‘valid pipelines whose end-to-end error factor is below one‘
‘stages_executed_before_failure_deepest‘	0.749	n	3631	–	‘stages run before an invalid depth-12 pipeline is caught at runtime‘

Artifact	SHA-256 (first 16)
‘artifacts/check_cost.json‘	‘762a5219a0d004df‘
‘artifacts/composition_validity.json‘	‘80db6aeb4ce7feeb‘
‘artifacts/error_propagation.json‘	‘30192ccf15aa078b‘

References

[1] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. *arXiv OpenAlex*, 2020.

[2] Sirui Hong, Mingchen Zhuge, Jiaqi Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, and Zili Wang. Metagpt: Meta programming for a multi-agent collaborative framework. *Crossref*, 2023.

[3] Sadam Hussain, Mansoor Ali, Farman Ali Pirzado, Masroor Ahmed, and José Gerardo Tamez-Peña. Comparative analysis of deep learning models for breast cancer classification on multimodal data. *Crossref*, 2024.

[4] Yixin Ji, Juntao Li, Yang Xiang, Hai Ye, Kaixin Wu, Kai Yao, Jia Xu, Linjian Mo, and Min Zhang. A survey of test-time compute: From intuitive inference to deliberate reasoning. *arXiv Crossref*, 2025.

[5] Eser Kandogan, Sajjadur Rahman, Nikita Bhutani, Dan Zhang, Rafael Li Chen, Kushan Mitra, Sairam Gurajada, Pouya Pezeshkpour, Hayate Iso, Yanlin Feng, Hannah Kim, Chen Shen, Jin Wang, and Estevam Hruschka. A blueprint architecture of compound ai systems for enterprise. *arXiv*, 2024.

[6] Andrew Konya, Deger Turan, Aviv Ovadya, Lina Qui, Daanish Masood, Flynn Devine, Lisa Schirch, Isabella Roberts, and Deliberative Alignment Forum. Deliberative technology for alignment. *arXiv*, 2023.

[7] Jiacheng Liang, Yuhui Wang, Changjiang Li, Rongyi Zhu, Tanqiu Jiang, Neil Gong, and Ting Wang. Graphrag under fire. *arXiv*, 2025.

[8] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan

Lowe. Training language models to follow instructions with human feedback. *arXiv OpenAlex*, 2022.

[9] Yoann Poupart. Contrastive sparse autoencoders for interpreting planning of chess-playing agents. *arXiv HAL*, 2024.

[10] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, and Weize Chen. Chatdev: Communicative agents for software development. *Crossref*, 2024.

[11] Ju Qin and Yuntao Sun. Fine-tuning clip with dynamic prompt tuning and cross-modal contrastive alignment for multimodal sentiment analysis. *Crossref*, 2026.

[12] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *arXiv OpenAlex*, 2023.

[13] Ashish Rana, Michael Oesterle, and Jannik Brinkmann. Gov-rek: Governed reward engineering kernels for designing robust multi-agent reinforcement learning systems. *arXiv*, 2024.

[14] Rasmus Ulfsnes, Nils Brede Moe, Viktoria Stray, and Marianne Skarpen. Transforming software development with generative ai: Empirical insights on collaboration and workflow. *arXiv*, 2024.

[15] Midhun Vayyat, Jaswin Kasi, Anuraag Bhattacharya, Shuaib Ahmed, and Rahul Tallamraju. Cluda : Contrastive learning in unsupervised domain adaptation for semantic segmentation. *arXiv*, 2022.

[16] Hao Yang, Jin Wang, and Xuejie Zhang. Dica: Dual-indicator guided contrastive alignment in multimodal large language models. *Crossref*, 2026.